HW Class 6 (R Functions)

Mikaela Joya (PID: A17295169)

Table of contents

Section 1: Improving analysis code by writing functions

A. Can you improve this analysis code?

```
#rescale01()
# Input: x (numeric vector)
# Function: Rescales the input vectors so all values are between 0 and 1
# To use: Use this function to a numeric vector
rescale01 <- function(x) {
  if (all(is.na(x))) {
    return(x)
}
  rng <- range(x, na.rm = TRUE)
  if (rng[1] == rng[2]) {
    return(rep(0, length(x)))
}
  (x - rng[1]) / (rng[2] - rng[1])
}
```

```
# Example usage
df <- data.frame(
  a = 1:10,
  b = seq(200, 400, length = 10),
  c = 11:20,
  d = NA
```

```
)

df[] <- lapply(df, rescale01)
```

B. Next improve the below example code for the analysis of protein drug interactions by abstracting the main activities in your own new function. Then answer questions 1 to 6 below.

Can you improve this analysis code?

```
library(bio3d)
```

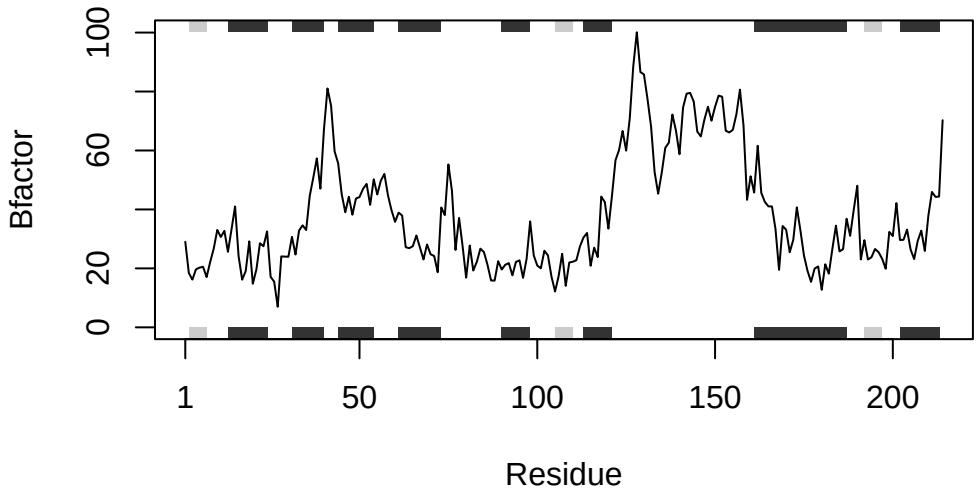
```
# analyze_protein()
# Input:
# pdb_id is a character string that represents PDB ID,
# chain_id is the protein chain used to analyze (default= "A"),
# atom_type is the atom type to select (default= "CA")
# Function:
# Reads the protein structure from the PDB database,
# trims to a specified chain and atom type,
# Then it extracts the B-factor values and plots the B-factors
# Output: Numeric vector with B-factor values for the selected atoms
analyze_protein <- function(pdb_id, chain_id = "A", atom_type = "CA") {
  pdb <- read.pdb(pdb_id)
  pdb_trim <- trim.pdb(pdb, chain = chain_id, elety = atom_type)
  b_factors <- pdb_trim$atom$b

  plotb3(b_factors, sse= pdb_trim, typ = "l", ylab = "Bfactor")

  return(b_factors)
}
```

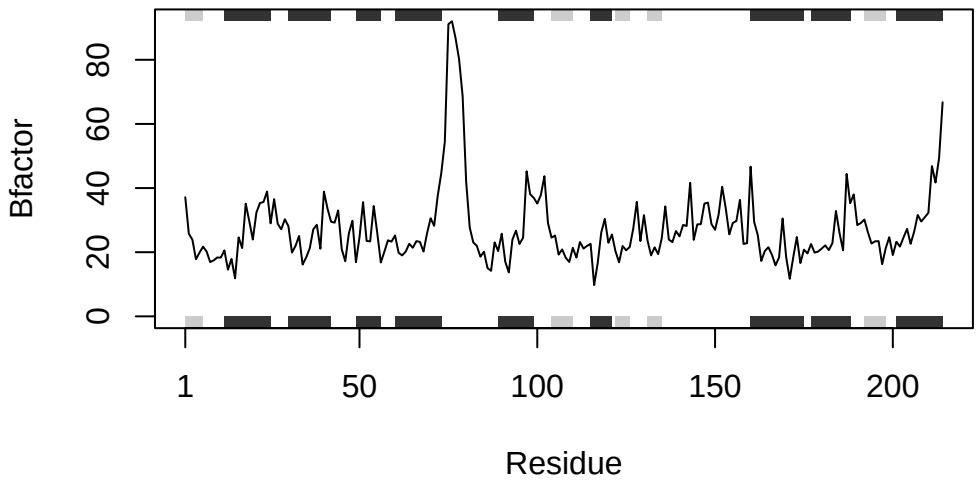
```
# Example usage
s1.b <- analyze_protein("4AKE") # kinase with drug
```

```
  Note: Accessing on-line PDB file
```



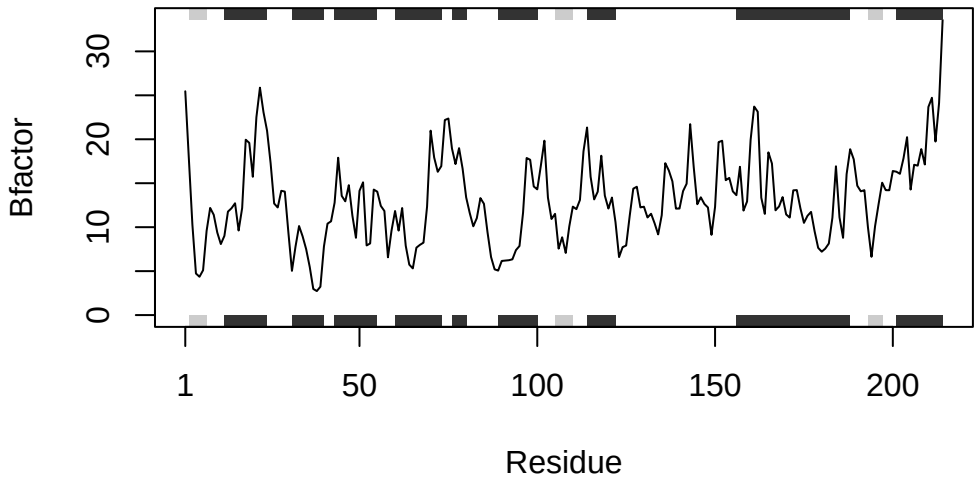

```
s2.b <- analyze_protein("1AKE") # kinase no drug
```

```
Note: Accessing on-line PDB file
 PDB has ALT records, taking A only, rm.alt=TRUE
```




```
s3.b <- analyze_protein("1E4Y") # kinase with drug
```

```
Note: Accessing on-line PDB file
```



Questions

Q1. What type of object is returned from the read.pdb() function?

The `read.pdb()` function returns a PDB object similar to a list that has information about the protein structure.

Q2. What does the trim.pdb() function do?

The `trim.pdb()` function selects only a portion of the PDB structure. This provides a simplified version of the structure if you need to use it for comparison. In Section B, the `trim.pdb()` function keeps only the chain A atoms and alpha carbons (CA).

Q3. What input parameter would turn off the marginal black and grey rectangles in the plots and what do they represent in this case?

`sse = NULL` or removing `sse` from `plotb3()` would turn fof the marginal black and grey rectangles in the plots. These black and grey retangles represent the secondary structure elements of the protein. In this case, they are the secondary structure for the `trim.pdb()` of chain A.

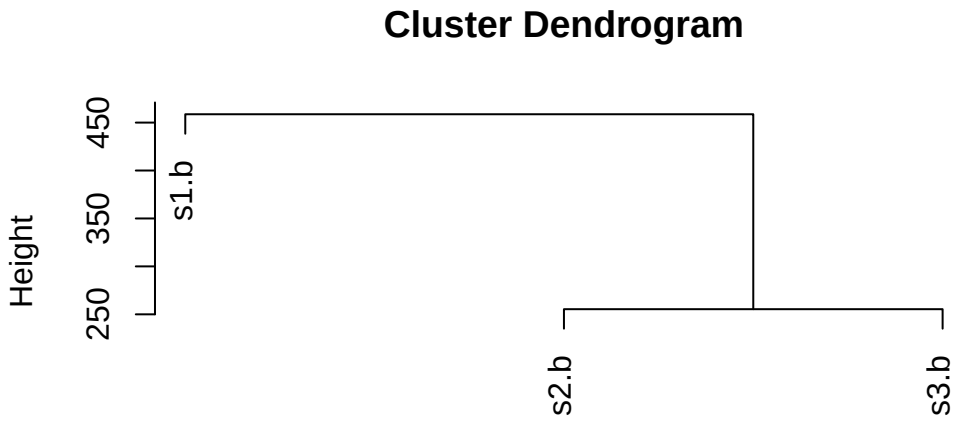
Q4. What would be a better plot to compare across the different proteins?

A better plot to compare different proteins would be one combined plot with all of the B-factor trends on the same graph with their appropriate labels.

Q5. Which proteins are more similar to each other in their B-factor trends. How could you quantify this?

The most similar proteins would be the ones with the most similar B-factor profiles. This could be quantified by putting the B-factor vectors together using `rbind()`, calculating the distances with `dist()`, and lastly clustering them `hclust()`.

```
hc <- hclust(dist(rbind(s1.b, s2.b, s3.b)))
plot(hc)
```



Based on the dendogram, the proteins 1AKE and 1E4Y (s2 and s3) are more similar to each other in their B-factor trends. This is because they cluster together at a lower height, while 4AKE (s1) is different since it joins the cluster at a higher distance.

> **Q6.** How would you generalize the original code above to work with any set of input protein structures?

The code given was generalized above. The code was generalized by writing a function that takes any PDB ID as an input, reading it, trimming it, extracting the B-factors, and plots the results as an output.